

Sparse Attention: Local + Global + Random.

Full self-attention costs $O(T^2)$ — every token looks at every other token. For long documents that is ruinous. Longformer and BigBird replace the dense map with a *sparse* one whose cost is $O(T)$, built from three ingredients: a local sliding window, a handful of global tokens, and a few random links. Every number below is computed in full.

THE CLAIM

$$\underbrace{O(T^2)}_{\text{dense}} \longrightarrow \underbrace{O(T(w + g + r))}_{\text{sparse, fixed } w, g, r} = O(T)$$

Worked example. $T = 4096$, window $w = 512$, $g = 4$ global tokens, $r = 3$ random links — we compute the exact cost reduction.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The cost of dense attention, and the fix	2
2	Three sparse components	3
3	Worked computation: $T = 4096$	3
4	The mask, drawn out ($T = 12$)	4
5	Properties & dilated windows	4
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The cost of dense attention, and the fix

Let a sequence have T tokens, each carried as a d -dimensional vector. Scaled dot-product attention forms the score matrix $S = QK^\top / \sqrt{d_k} \in \mathbb{R}^{T \times T}$, applies a row-wise softmax, and multiplies by V :

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1)$$

The bottleneck is $QK^\top$: it has T^2 entries and costs $T^2 d_k$ multiply-adds, plus T^2 memory to store S . Both the time *and* the memory grow *quadratically* in T .

Most tokens do not need to talk to most other tokens. Language is overwhelmingly local (a word's meaning is set by its neighbours), with a few long-range anchors (a title, a question, a topic sentence). Sparse attention hard-codes this prior: keep the local links, add a small number of long-range ones, and throw the rest away. The number of links kept per token is a *constant*, so total work is $O(T)$.

The sparse pattern is a binary mask $M \in \{0, 1\}^{T \times T}$ with $M_{ij} = 1$ iff query i is allowed to attend to key j . We compute scores only on the allowed entries:

$$S_{ij} = \begin{cases} \frac{q_i \cdot k_j}{\sqrt{d_k}}, & M_{ij} = 1 \\ -\infty, & M_{ij} = 0 \end{cases} \quad \text{cost} \propto \text{nnz}(M).$$

The whole game is to make $\text{nnz}(M)$ scale like T rather than T^2 .

2 Three sparse components

(a) Local sliding window (Longformer). Each token attends to a window of width w centred on itself: $M_{ij} = 1$ iff $|i - j| \leq w/2$. Every row has $\leq w$ nonzeros, so

$$\text{nnz}_{\text{local}} = T \cdot w = O(Tw).$$

(b) Global tokens. Designate g special tokens (e.g. [CLS], the question in QA). A global token attends to *all* keys, and is attended by *all* queries: it fills g full rows and g full columns. That adds

$$\text{nnz}_{\text{global}} = 2gT - g^2 \approx 2gT = O(gT).$$

(c) Random links (BigBird). Each token additionally attends to r keys chosen uniformly at random, adding

$$\text{nnz}_{\text{random}} = T \cdot r = O(Tr).$$

Summing, with w, g, r held constant as T grows:

$$\boxed{\text{nnz}(M) = Tw + 2gT + Tr = T(w + 2g + r) = O(T)} \quad (2)$$

Quadratic $\rightarrow$ linear is purely a matter of bounding the number of nonzeros *per row* by a constant. Local handles nearby context, global injects sequence-wide summaries, random provides cheap long-range shortcuts. BigBird = local + global + random; Longformer = local + global ($r = 0$).

3 Worked computation: $T = 4096$

Take $T = 4096$, $w = 512$, $g = 4$.

Longformer ($r = 0$).

$$\begin{aligned} \text{dense} &= T^2 = 4096^2 = 16\,777\,216, \\ \text{local} &= Tw = 4096 \cdot 512 = 2\,097\,152, \\ \text{global} &= 2gT = 2 \cdot 4 \cdot 4096 = 32\,768, \\ \text{sparse} &= 2\,097\,152 + 32\,768 = 2\,129\,920. \end{aligned}$$

Reduction factor = $16\,777\,216 / 2\,129\,920 = 7.88\times$ — the score matrix shrinks to $\approx 12.7\%$ of dense.

BigBird ($r = 3$). Add $Tr = 4096 \cdot 3 = 12\,288$:

$$\text{sparse} = 2\,129\,920 + 12\,288 = 2\,142\,208, \quad \text{reduction} = 16\,777\,216 / 2\,142\,208 = 7.83 \times .$$

The random links cost almost nothing (0.6% of the budget here) yet, as Topic 2 shows, they are what make the sparse graph mix globally.

Cost across sequence lengths ($w = 512$, $g = 4$, $r = 0$)

T	dense T^2	sparse $Tw+2gT$	reduction
512	262 144	266 240	0.98×
1 024	1 048 576	532 480	1.97×
4 096	16 777 216	2 129 920	7.88×
16 384	268 435 456	8 519 680	31.5×
65 536	4 294 967 296	34 078 720	126×

The crossover: below $T \approx w$ the window covers the whole sequence and “sparse” is no cheaper than dense (the $T = 512$ row even costs slightly *more* because of the global columns). The win grows linearly with T : doubling T roughly doubles the reduction factor.

4 The mask, drawn out ($T = 12$)

A toy mask combining a local window (half-width 1, so width $w = 3$), one global token (token 0), and one random link per non-global token (picks $2 \rightarrow 7$, $3 \rightarrow 9$, $5 \rightarrow 10$, $6 \rightarrow 11$, $8 \rightarrow 3$, $9 \rightarrow 4$, $11 \rightarrow 6$). Rows are queries i , columns are keys j ; **indigo** = allowed ($M_{ij} = 1$), light = masked. Token 0 fills its whole row and column.

	0	1	2	3	4	5	6	7	8	9	10	11
0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	1	1	0	0	0	0	0	0	0	0	0
2	1	1	1	1	0	0	0	1	0	0	0	0
3	1	0	1	1	1	0	0	0	0	1	0	0
4	1	0	0	1	1	1	0	0	0	0	0	0
5	1	0	0	0	1	1	1	0	0	0	1	0
6	1	0	0	0	0	1	1	1	0	0	0	1
7	1	0	0	0	0	0	1	1	1	0	0	0
8	1	0	0	1	0	0	0	1	1	1	0	0
9	1	0	0	0	1	0	0	0	1	1	1	0
10	1	0	0	0	0	0	0	0	0	1	1	1
11	1	0	0	0	0	0	1	0	0	0	1	1

The three bands are visible: a tridiagonal stripe (local), the full first row/column (global token 0), and scattered off-band cells (random). This mask has 61 allowed entries of 144 — but note that for tiny T the single global token alone fills $2T - 1 = 23$ cells, which is why density looks high; at $T = 4096$ the same pattern is $\approx 12.7\%$ dense.

5 Properties & dilated windows

1. **Linear cost.** Time and memory are $O(T(w + 2g + r)) = O(T)$ for fixed w, g, r .
2. **Drop-in.** The mask multiplies into the existing attention; weights and the softmax/ V steps are unchanged. Only the score computation is restricted.
3. **Receptive field grows with depth.** One local layer reaches $\pm w/2$; stacking L layers reaches $\pm Lw/2$ (Topic 2 quantifies how many layers global mixing needs).
4. **Dilated windows (Longformer).** To enlarge the receptive field *without* raising w , leave gaps in the window: token i attends to j with $i - j \in \{\pm d, \pm 2d, \dots\}$ for dilation d . A window of w slots with dilation d spans $d w$ positions at the same cost. Different heads use different d (small d for local detail, large d for reach).

The reduction is only real for $T \gg w$. If you set $w = 512$ but your documents are 512 tokens long, the window covers everything and you have paid for masking with no speedup — the $T = 512$ row above is actually *slower* than dense. Sparse attention is a long-sequence tool; for short inputs, dense is both simpler and faster.

6 Where this sits in the track

Direction	Topic	Link
Back	Scaled dot-product attention	the $O(T^2)$ we are fixing
Here	Sparse local+global+random	make $\text{nnz}(M) = O(T)$
Next	Expander-graph mixing	<i>why</i> the random links suffice

A Reproducible (NumPy)

Inputs: $T = 4096$, $w = 512$, $g = 4$, $r \in \{0, 3\}$; toy mask $T = 12$, half-width 1, global token 0.

```
import numpy as np
T,w,g = 4096,512,4
dense = T*T # 16777216
local = T*w # 2097152
glob = 2*g*T # 32768
sparse = local + glob # 2129920 (Longformer, r=0)
print(dense, sparse, dense/sparse) # 16777216 2129920 7.8769...
bb = sparse + T*3 # 2142208 (BigBird, r=3)
print(dense/bb) # 7.8317...

# cost table
for T in [512,1024,4096,16384,65536]:
    print(T, T*T, T*w+2*g*T, round(T*T/(T*w+2*g*T),2))
# 512 262144 266240 0.98 ; 4096 16777216 2129920 7.88 ; 65536 ... 126.0

# toy mask T=12
T,half = 12,1
M = (np.abs(np.subtract.outer(range(T),range(T))) <= half).astype(int)
M[0,:] = 1; M[:,0] = 1 # global token 0
for i,j in {2:7,3:9,5:10,6:11,8:3,9:4,11:6}.items():
    M[i,j] = 1 # random links
print(M.sum()) # 61
```